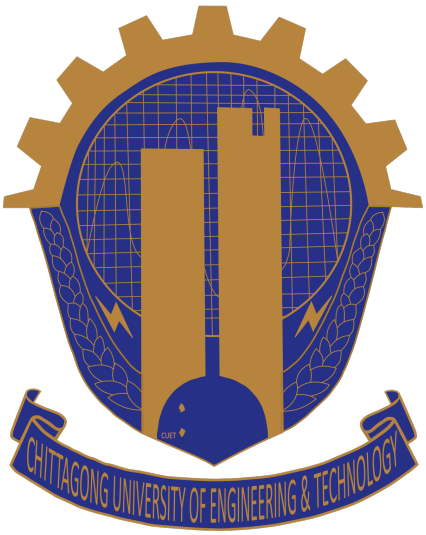
Internet Programming (sessional) [CSE-326]



Proposal Report on:

Hall Complaint Management

Group No: A1-02

Name	Student ID
Ahmed Abrar Zayad	2204021
Shahneous Ali	2204013
Fardin Ahmed	2204014

Submitted to:

Md. Rashadur Rahman

Assistant Professor,

Department of CSE, CUET

Department of Computer Science and Engineering (CSE)

Chittagong University of Engineering & Technology (CUET)

Chattogram-4349, Bangladesh

1 Executive Summary

A full-stack web application called the Complaint Management System for CUET is developed to digitize and speed the complaint management procedure in university residence halls. The system substitutes an organized, effective, and transparent digital platform for conventional manual procedures.

Administrators can monitor, prioritize, and address concerns via a unified dashboard, while students can file complaints, monitor their status, and handle their problems. A full complaint lifecycle, including Pending, Unresolved, and Solved statuses, is enforced by the system.

Key features include:

- Role-based authentication (Student and Admin)
- Complaint submission with file upload
- Real-time dashboard and statistics
- Complaint lifecycle management
- Student management per hall
- Report generation with PDF export

The system is built using Next.js for frontend, FastAPI for backend, and PostgreSQL as the database. JWT-based authentication ensures secure access control.

2 Project Recap and Requirements Analysis

2.1 Objectives

This project's main objective was to create a digital system that would effectively handle complaints in CUET residential halls, cutting down on delays and enhancing accountability. The system substitutes a more organized and transparent method for the conventional manual technique.

Students may quickly file complaints and follow their progress, while administrators can use a central dashboard to keep an eye on and address problems. The system's overall goals include greater organization, quicker reaction times, and more communication between administrators and students.

2.2 Motivation

Manual complaint processes frequently result in delays, missing information, and poor communication between students and administrators because they are difficult, time-consuming, and ineffective. By implementing structured complaint resolution procedures and real-time tracking, this system resolves these problems. It guarantees that every complaint is appropriately documented, tracked, and handled in a methodical way. Consequently, the system increases overall efficiency and user experience, decreases reaction time, and increases transparency.

2.3 Stakeholders

- Students: Submit and track complaints
- Hall Provost : Manage complaints and monitor system

2.4 Feature Implementation Table

Feature	Status
Authentication System	Implemented
Complaint Submission	Implemented
Complaint Tracking	Implemented
Admin Dashboard	Implemented
Student Management	Implemented
PDF Report Generation	Implemented
File Upload Support	Implemented
Email Notification	Not Implemented
Push Notifications	Not Implemented
Dark Mode	Not Implemented

Reasons for Not Implemented Features:

- **Email Notification:** Requires third-party email service integration, which involves additional cost and configuration. Due to limited resources, this feature was not included.
- **Push Notifications:** Since users can monitor updates directly from the system dashboard, push notifications were not considered necessary at this stage.
- **Dark Mode:** This is a user interface enhancement and was not prioritized over core system functionalities.

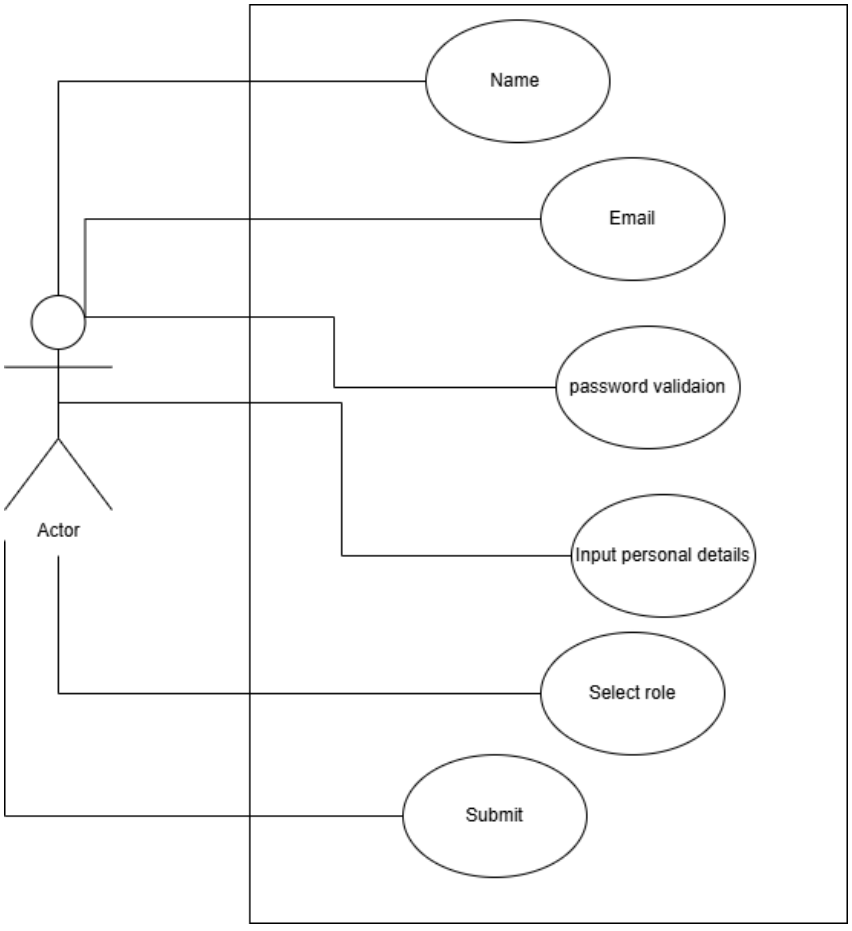
2.5 Updated Stakeholder Analysis

The system successfully improves user experience by providing transparency and control to both students and administrators.

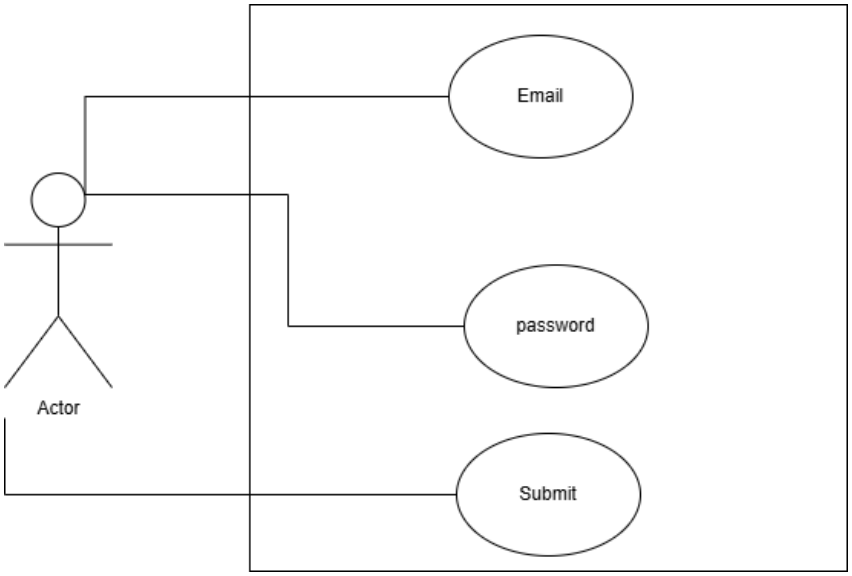
3 System Architecture and Design

3.1 Architecture Diagram

This is the UML diagram for registration. Here, user first inputs name, email and password. Then they input personal details and select their role. Then they submit their information for creating an account.

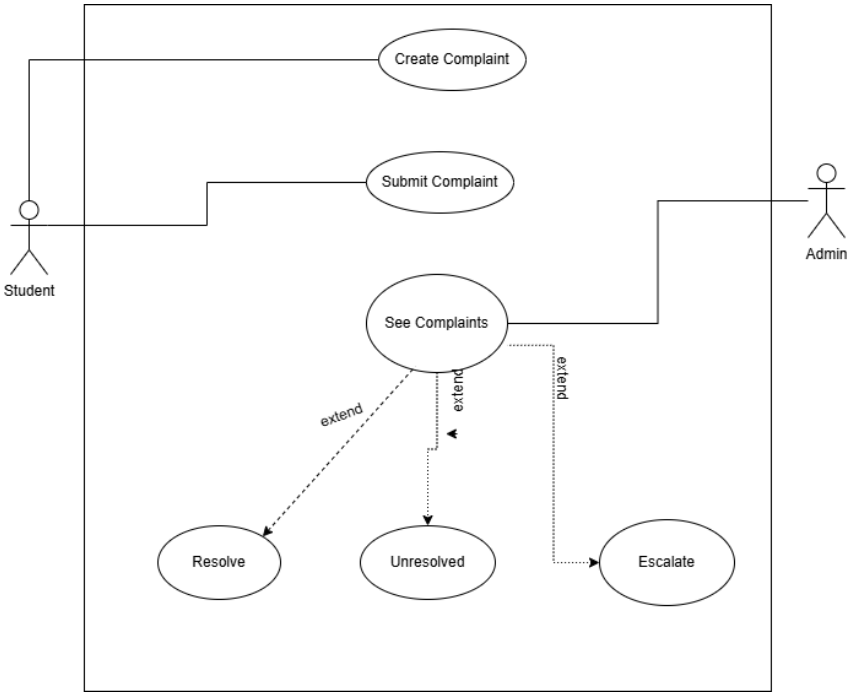


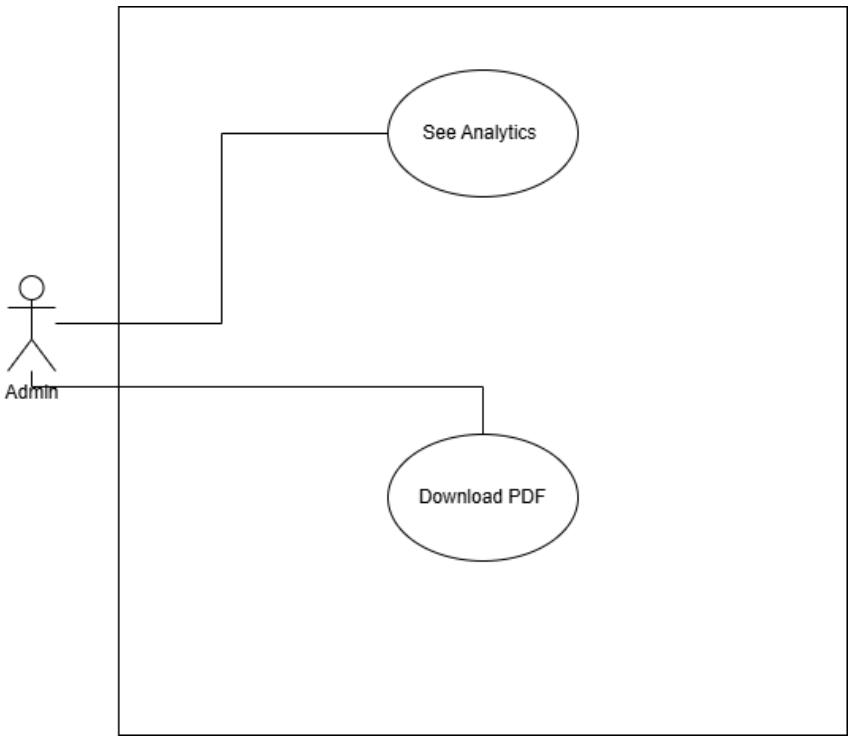
This is the UML diagram for login. Here, user first inputs email and password. Then they submit their information for logging in with their account. If they are student, they go to the student portal. Or else, they go to the admin portal.



This is the UML diagram for complaints. Here, student creates complaints and then the students submit the complaints.

The admin sees the complaints, and they can resolve, unresolve, and escalate the complaints.





This use case diagram illustrates the report generating portal for the admins. They can see the analytics related to the reports and they can download the report.

Actor:

- Admin

Use Cases:

- Login: Admin accesses the system using registered credentials.
- View Dashboard & Statistics: Admin views aggregated statistics such as total, pending, unresolved, and resolved complaints.
- Manage All Complaints: Admin can browse, search, and filter all complaints.
- Update Complaint Status: Admin marks complaints as solved or unresolved.
- Manage Students: Admin manages student accounts within the hall.
- View Reports & Analytics: Admin views reports and complaint trends.
- Download PDF Report: Admin exports reports as PDF.

Student Use Case Diagram :

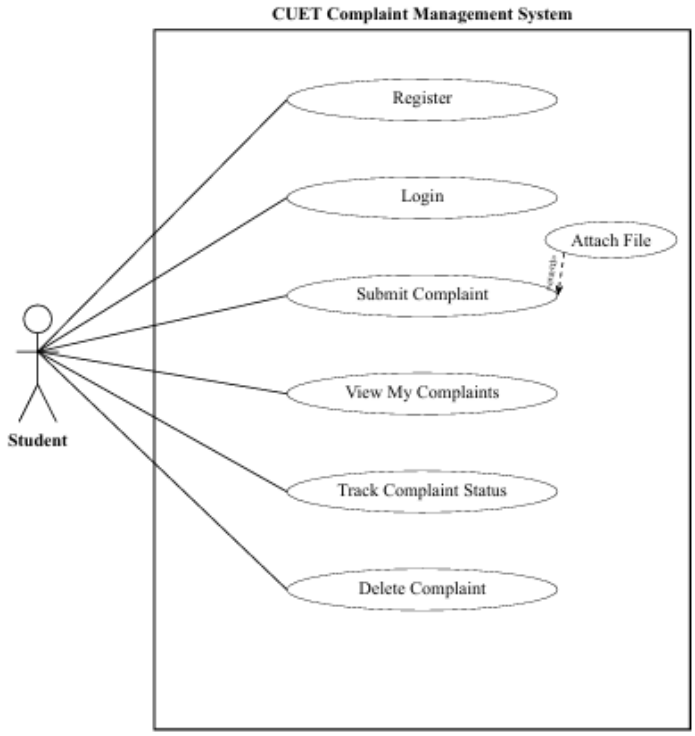


Figure 2: Student Use Case Diagram – CUET Complaint Management System

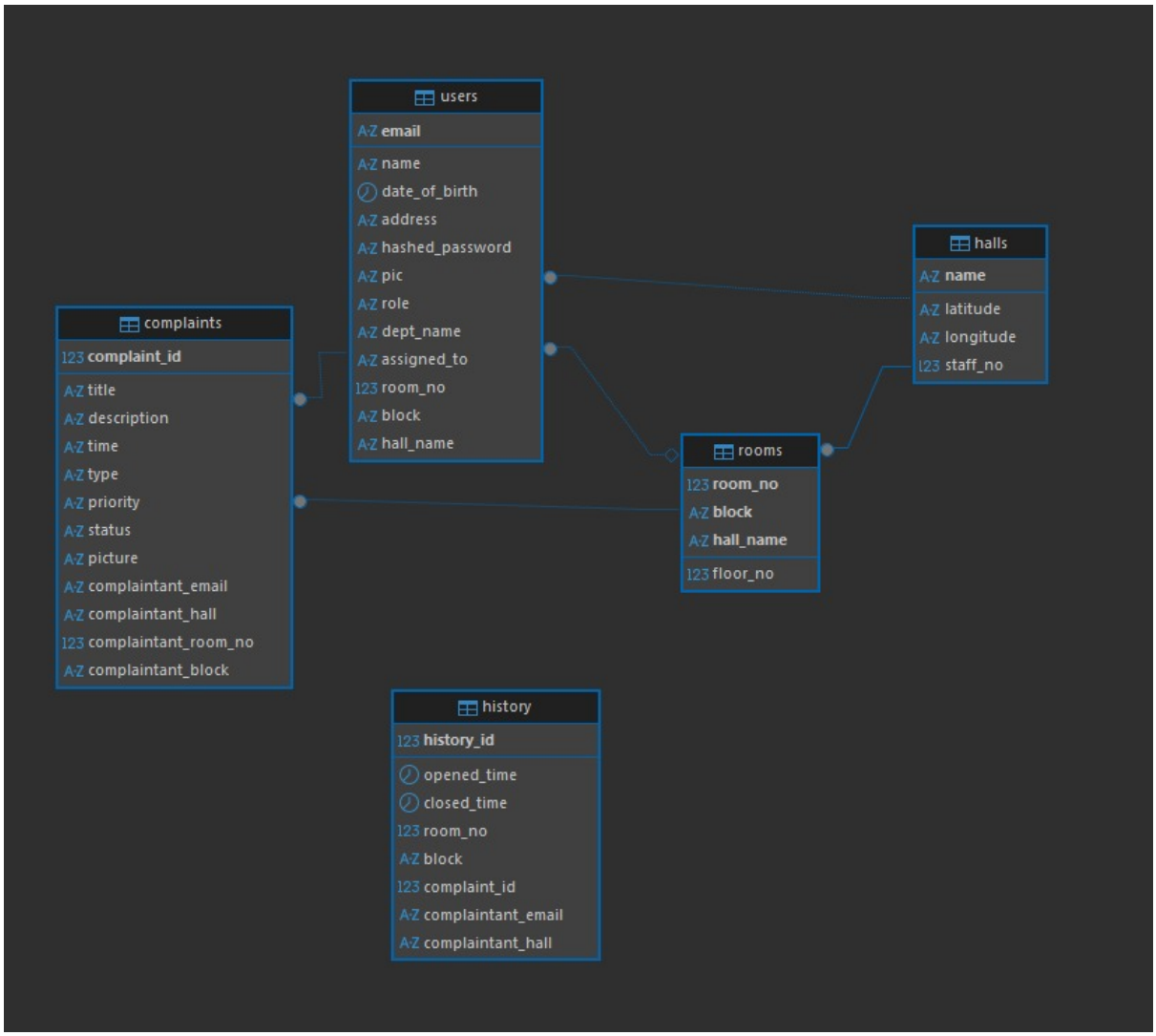
This use case diagram defines the interactions available to a registered student. Students can manage their own complaints but cannot access system-wide controls. **Actor:**

- Student

Use Cases:

- Register: Student creates a new account.
- Login: Student logs into the system.
- Submit Complaint: Student submits a complaint with details.
- Attach File: Optional file upload during complaint submission.
- View My Complaints: Student views their complaints.
- Track Complaint Status: Student monitors complaint status.
- Delete Complaint: Student deletes their own complaint.

3.2 Entity Relationship Diagram



The Complaint Management System's database structure is depicted in the Entity Relationship (ER) diagram. Users, complaints, rooms, halls, and history are its five primary entities. While the complaints table keeps track of every complaint that has been filed, along with its details and status, the users table holds information about students and administrators. Every complaint is associated with a certain room and user. The physical structure of the residential system, where rooms correspond to halls, is defined by the rooms and halls tables. The complaint lifecycle, including opening and closing times, is tracked in the history table. All things considered, the connections guarantee appropriate data structure, allowing for effective complaint tracking and management.

THe users and halls have many to one relationship. Rooms and halls have many to one relationship. Users and rooms have many to one relationship. Complaints and users have one to many relationship. Complaints have many to one relationship.

3.3 Database Design Description

The database consists of:

- Users
- Complaints
- Rooms
- Halls
- History

Each user can create multiple complaints. Complaints are linked with rooms and halls, and history tracks lifecycle events.

3.4 Technology Choices

- Next.js: Modern frontend framework
- FastAPI: High-performance backend
- PostgreSQL: Reliable relational database
- JWT: Secure authentication

3.5 API Endpoints

Method	Endpoint	Purpose
POST	/auth/login	User login
POST	/complaints	Create complaint
GET	/complaints	Fetch complaints
PATCH	/complaints/{id}	Update status
GET	/dashboard/admin-stats	Dashboard data
GET	/history	Complaint history

4 Backend Implementation

4.1 Server-side Architecture and Logic

The backend is implemented using FastAPI with a layered architecture:

- **Router layer:** Defines API endpoints and dependency injection.
- **Service layer:** Handles business logic such as authentication, complaint lifecycle, and analytics.

- **Repository layer:** Encapsulates SQLAlchemy database operations.
- **DTO layer:** Uses Pydantic models for request and response validation.
- **Security layer:** Implements password hashing, JWT generation, and role checks.

Core application setup:

```
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.mount("/pictures", StaticFiles(directory="pictures"), name="pictures")
app.include_router(auth_router)
app.include_router(dashboard_router)
app.include_router(complaints_router)
app.include_router(lookups_router)
app.include_router(students_router)
app.include_router(history_router)
```

4.2 Authentication and Authorization

Authentication uses JWT with access and refresh tokens:

- Passwords are hashed with bcrypt via Passlib.
- Access token and refresh token are issued at login.
- Protected routes use bearer token validation.
- Admin-only endpoints are enforced via role dependency checks.

Token generation logic:

```
def create_access_token(data: dict) -> str:
    payload = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload.update({"exp": expire, "type": "access"})
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=settings.
   ALGORITHM)
```

```
def create_refresh_token(data: dict) -> str:
    payload = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        days=settings.REFRESH_TOKEN_EXPIRE_DAYS
    )
    payload.update({"exp": expire, "type": "refresh"})
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=settings.
    ALGORITHM)
```

Authorization dependencies:

```
def get_current_user(...):
    payload = decode_token(credentials.credentials)
    email: str = payload.get("sub")
    user = db.query(Users).filter(Users.email == email).first()

    if user is None:
        raise HTTPException(status_code=401, detail="User not found")

    return user

def get_admin_user(current_user: Users = Depends(get_current_user)) -> Users:
    if current_user.role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")

    return current_user
```

4.3 CRUD Operations and Business Logic

Complaint module supports full lifecycle operations:

- **Create complaint:** Accepts multipart data with optional image upload.
- **Read complaints:** Student can fetch own complaints; admin can fetch all.
- **Update status:** Admin updates complaint to `solved` or `unresolved`.
- **Delete complaint:** Student can delete only their own complaint.
- **History tracking:** A history entry is created when complaint is opened and updated when solved.

Create complaint route:

```
@router.post("/", response_model=ComplaintResponseDTO, status_code=201)
async def create_complaint(
    title: str = Form(...),
    type: ComplaintType = Form(...),
    description: str = Form(...),
    priority: ComplaintPriority = Form(...),
    picture: Optional[UploadFile] = File(None),
    current_user: Users = Depends(get_current_user),
    service: ComplaintsService = Depends(get_complaints_service),
):
    data = CreateComplaintDTO(
        title=title,
        type=type,
        description=description,
        priority=priority,
    )
    picture_data = (
        await picture.read(),
        picture.filename,
    ) if picture and picture.filename else None

    return service.create_complaint(data, current_user, picture_data)
```

Business logic example:

```
complaint = Complaints(
    title=data.title,
    type=data.type.value,
    description=data.description,
    priority=data.priority.value,
    status=ComplaintStatus.PENDING.value,
    time=datetime.now(timezone.utc).isoformat(),
    picture=picture_path,
    complaintant_email=current_user.email,
    complaintant_hall=current_user.assigned_to,
    complaintant_room_no=current_user.room_no,
    complaintant_block=current_user.block,
)

saved = self.repo.create(complaint)
self.history_repo.create_entry(...)
```

4.4 Database Schema Creation

Schema is managed through Alembic migrations:

- Initial migration creates `halls`, `rooms`, `users`, `complaints`, and `history`.
- Composite keys enforce room uniqueness per hall using `(room_no, block, hall_name)`.
- A later migration makes student residence fields nullable for non-student users.

Key migration files:

- `alembic/versions/910bce1bdf6f_initial_schema.py`
- `alembic/versions/c2a1d7f4b9d8_make_user_residence_nullable.py`

4.5 Sample Data Population

Seed scripts are used for hall and room data:

- Data source: `seed_data/halls.json` and `seed_data/rooms.json`.
- Import script: `scripts/import_data.py`.
- Normalizes hall naming variants to enum-compliant values.
- Idempotent behavior: updates existing rows and inserts missing rows.

4.6 Complex Queries with Explanations

The backend includes reporting and aggregation queries:

1. **Admin dashboard aggregation:** Counts total, pending, unresolved, and resolved complaints.
2. **Daily filtering:** Fetches today's pending and unresolved complaints using UTC date windows.
3. **History analytics:** Computes total, resolved, pending, average resolution hours, and monthly opened vs resolved trends.

Dashboard query example:

```
number_of_pending_complaints = (
    self.db.query(Complaints)
    .filter(Complaints.status == ComplaintStatus.PENDING.value)
    .count()
)
```

```
list_of_pending_complaints = (
    self.db.query(Complaints)
    .filter(
        Complaints.status == ComplaintStatus.PENDING.value,
        Complaints.time >= today_start,
        Complaints.time < today_end,
    )
    .order_by(Complaints.time.desc())
    .all()
)
```

4.7 Validation and Error Management

Validation strategy:

- Pydantic DTO validation for required fields, password constraints, and enum-safe inputs.
- Domain validation to ensure students provide valid hall, block, and room information.
- Ownership and role checks for admin-only actions and owner-only delete or read access.

Error handling strategy:

- `400 Bad Request`: invalid domain input.
- `401 Unauthorized`: invalid or expired token, invalid login.
- `403 Forbidden`: role mismatch or ownership violation.
- `404 Not Found`: missing resources.
- `409 Conflict`: duplicate registration when email already exists.

4.8 Backend Dashboards, Interfaces, and API Responses

Major dashboard and report interfaces exposed by backend:

- `GET /dashboard/admin-stats`
- `GET /history/`
- `GET /history/stats`

- `GET /students/{hall_name}/students-list`

  Example API response:

```
{
  "total_complaints": 42,
  "pending_complaints": 12,
  "unresolved_complaints": 8,
  "resolved_complaints": 22,
  "today_pending_complaints": [],
  "today_unresolved_complaints": []
}
```

5 Frontend Implementation and Integration

5.1 Frontend Structure for Major Pages

Frontend is built with Next.js App Router and a container/view split:

- **Authentication:**
  - Login page: `app/(auth)/page.tsx`
  - Register page: `app/(auth)/register/page.tsx`, which contains a multi-step form.
- **Student area:**
  - Submit complaint
  - My complaints with filter, pagination, and details
- **Admin area:**
  - Dashboard
  - Manage complaints
  - Manage students
  - Reports with charts and PDF export

5.2 JavaScript API Calls, Data Binding, and State Management

Data integration approach:

- API service modules in `lib/`: `complaints.ts`, `history.ts`, `students.ts`, and `lookups.ts`.
- Axios instance via custom hook `useAxios.tsx`.
- NextAuth session stores user role and access token.
- React state management uses `useState`, `useEffect`, `useMemo`, and `useCallback`.

Axios auth integration:

```
const requestInterceptor = axiosInstance.interceptors.request.use((config) =>
   {
  const accessToken = session?.accessToken;

  if (accessToken) {
    config.headers.Authorization = `Bearer ${accessToken}`;
  }

  return config;
});
```

5.3 Form Validation and User Feedback

Implemented validation and feedback includes:

- Login validation with backend-authenticated role and credentials.
- Registration checks for required steps, password match, and hall/block/room consistency.
- Complaint submission checks for required title, category, description, priority, and optional file upload.
- Inline error messages and disabled loading buttons during submission.

5.4 Dynamic Content Loading

Dynamic UI behavior:

- Complaint tables load after authenticated session.
- Registration hall and room dropdowns load dynamically from backend.
- Reports page fetches history and aggregated stats in parallel.
- Updates such as status change and delete actions reflect immediately in local UI state.

5.5 Responsive Design, Media Queries, and Mobile UX

Responsive behavior is handled via Tailwind utility breakpoints:

- Breakpoints used: `sm`, `md`, `lg`, and `xl`.
- Sidebar has mobile toggle menu with overlay.
- Tables are wrapped in `overflow-x-auto` for narrow screens.
- Grid layouts adapt from single-column to multi-column views.

5.6 Breakpoints Tested and Mobile Considerations

Typical responsive patterns present in components:

- `grid-cols-1 sm:grid-cols-2 xl:grid-cols-4`
- `lg:ml-64` for desktop sidebar spacing
- `lg:hidden` for mobile menu toggle controls
- Form cards and actions stack vertically on smaller viewports.

5.7 Before/After Styling and Interactive States

UI interaction states are clearly designed:

- **Default state:** neutral border/background with readable contrast.
- **Hover state:** elevated shadow, color emphasis, and subtle movement.
- **Focus state:** visible focus rings for accessibility.
- **Active/selected state:** tab selection, sidebar active links, and selected priority buttons.
- **Modal state:** keyboard `Escape` close and overlay click close.

5.8 Frontend-Backend Integration Summary

Integration is consistent and role-aware:

- NextAuth authenticates through backend `/auth/login`.
- Backend JWT token is attached automatically in Axios interceptor.
- Role-based UI actions align with backend permission checks.
- Dashboard and report pages consume aggregated backend endpoints directly.

6 Evaluation and Reflection

6.1 Comparison with Proposal

The system successfully implements most of the proposed features including authentication, complaint management, and reporting.

6.2 Challenges Faced

- JWT authentication integration
- File upload handling
- Composite key management
- Role-based routing

6.3 Performance Analysis

The system performs efficiently under normal conditions. API responses are fast and UI updates dynamically.

6.4 Limitations

- No real-time notifications
- No email alerts
- Limited UI customization

7 Conclusion and Contributions

The project effectively illustrates full-stack development abilities, such as database administration, backend architecture, and frontend design. It offers a workable alternative for managing complaints in university buildings, increasing effectiveness and openness. We used FastAPI for backend, sqlalchemy for defining models, and alembic for migrations. We used postgreSQL for database and used Next.ts for the frontend. We initially created the UI using HTML,CSS,JS. This project aims to create a digital system for complaint management for halls.

7.1 Learning Outcomes

- Full-stack development experience
- API design and integration
- Authentication and security implementation
- Database modeling

7.2 Self Evaluation

The system meets most objectives and provides a scalable foundation for future improvements. We have achieved the goals we set out to achieve with this system.

8 Appendices

8.1 Source Code

`https://github.com/FardinAhmed-ScriptedSoul/Complaint-Management-System`

8.2 Database

Database schema and seed data are included in the repository.